

نظام استرجاع المعلومات

اعداد الطلاب:

محمد ارناؤوط

محمد خير سلام

سارية باكير اغا

احمد مضر بيازيد

رياض الفرحان

م. مروة الداية

الجزء الأول — نظرة عامة على النظام

هذا المشروع عبارة عن نظام استرجاع معلومات (\$IR\$) كامل ومجزأ (\$Modular\$) ، مبني حول مكتبة محرك مشتركة موحدة (ir_core) يتم تقديمها بثلاث طرق: واجهة مستخدم ويب Streamlit ، وبنية موجهة للخدمات (\$SOA\$) مكونة من خمس خدمات مصغرة (\$Microservices\$) مبنية بإطار عمل FastAPI وتقع خلف بوابة واجهة برمجية التطبيقات (\$API\ Gateway\$) ، ومجموعة من السكريبتات البرمجية لواجهة السطر البرمجي (\$CLI\$) لبناء الفهارس وتشغيل عمليات التقييم.

يقوم النظام بالاسترجاع عبر مجموعة وثائق المناظرات والحجج (Args.me) تضم 387,692 وثيقة، لمهمة Touché 2020 الأولى، حيث يتم سحب استعلامات التقييم وأحكام الصلة (\$Relevance\ Judgments\ \ qrels\$) من مكتبة ir_datasets. كما تتضمن الشيفرة البرمجية عينة مصغرة تحتوي على 5 وثائق فقط مخصصة لاختبارات الدخان. (\$Smoke\ Tests\$)

يقوم النظام بتطبيق ستة نماذج استرجاع خلف واجهة برمجية موحدة:

النموذج	التمثيل (Representation)	المطابقة والتطابق (Matching)
TF-IDF (VSM)	أوزان حدودية خفيفة (\$Sparse\$) من نوع ltc	تشابه جيب التمام (\$Cosine\ Similarity\$)
BM25	نموذج Okapi الاحتمالي، مع معملين قابلين للضبط b و k1	نتيجة ومعدل BM25

المطابقة والتطابق (Matching)	التمثيل (Representation)	النموذج
جيب التمام (\$Cosine\$)	تضمينات الجمل باستخدام نموذج all-MiniLM-L6-v2	BERT
جيب التمام (\$Cosine\$)	نموذج skip-gram عبر PyTorch ، مع تجميع متوسط ناقلات الكلمات	Word2Vec
الجمع الموزون- (\$Weighted\$ sum) أو دمج الرتب المتبادلة (\$RRF\$)	يجمع نماذج متعددة	الهجين المتوازي (Parallel)
نتيجة المرحلة الثانية	نموذج أول يقوم بالاسترجاع، ونموذج ثانٍ يعيد الترتيب (\$Re-ranks\$)	الهجين المتسلسل (Serial)

الجزء الثاني — تغطية المتطلبات

تم تنظيم الشيفرة البرمجية (Codebase) لتغطي المتطلبات التسعة المحددة في موجز المشروع:

#	المتطلب	مكان تواجده في الكود
2	التمثيلات: نماذج VSM TF-IDF ، و BM25 ، والتضمينات (BERT) و (Word2Vec) ، والنظام الهجين (المتسلسل و المتوازي مع الدمج)	ir_core/representation/
3	الفهرسة: فهرس مقلوب مدعوم على القرص الصلب (SQLite) ، مع بناء تدفقي (\$Streamed\ Build\$)	ir_core/index/inverted_index.py
4	معالجة الاستعلام: معالجة أولية متطابقة للوثائق والاستعلامات على حد سواء	ir_core/text/preprocessing.py, ir_core/query/processing.py
5	تحسين الاستعلام: تصحيح الأخطاء الإملائية، التوسيع باستخدام WordNet ، الاقتراحات، وسجل البحث	ir_core/query/refinement.py

#	المتطلب	مكان تواجده في الكود
6	المطابقة والترتيب: جيب التمام / BM25 / حاصل الضرب النقطي (\$Dot\ Product\$) لكل نموذج	ir_core/representation/*, ir_core/engine.py
7	البنية الموجهة للخدمات: (SOA) خمس خدمات مصغرة + بوابة API باستخدام FastAPI/REST)	services/
8	التقييم: مقاييس MAP ، Recall ، و P@10 ، و nDCG@10 ، قبل وبعد تحسين الاستعلام	ir_core/eval/
9	واجهة المستخدم: تطبيق ويب مبني بـ Streamlit	ui/app.py

الجزء الثالث — البنية الهندسية (\$Architecture\$)

الأنماط التصميمية المستخدمة: (\$Design\ Patterns\$) يمثل صنف Engine واجهة موحدة (Facade) فوق جميع التمثيلات وأدوات التحسين. ينفذ كل مسترجع (سواء كان لغظياً \$Lexical\$ أو كثيفاً \$Dense\$) نفس الواجهة القائمة على نمط الاستراتيجية (Strategy) التابعين (search/rescore) بحيث يتعامل المحرك والنماذج الهجينة معها بشكل موحد. بوابة واجهة البرمجيات (API Gateway) تعمل كبوابة وواجهة موحدة لا تحتوي على أي منطق أعمال — (\$Business\ Logic\$) بل تقوم بتوجيه كل استدعاء إلى الخدمة المالكة له ودمجها عند الحاجة (على سبيل المثال، الرابط search_refined/يسندعي خدمة التحسين أولاً ثم خدمة الاسترجاع). تم استخدام نمط المفردة (Singleton) للكائن ENGINE على مستوى العملية بأكملها للحفاظ على الفهارس المحملة دافئة وجاهزة عبر الطلبات المتعددة.

واجهة مستخدم واحدة لكلا النظامين: يمكن لتطبيق Streamlit استدعاء المحرك داخلياً ضمن نفس العملية (بدون خوادم) أو توجيه الطلبات عبر البوابة (SOA) ؛ حيث يتيح شريط جانبي التبديل بينهما، ويقدم كلاهما العمليات المتطابقة تماماً.

الجزء الرابع — كيف تعمل عملية البحث (من البداية إلى النهاية)

1. تقوم واجهة المستخدم (أو عميل البوابة) باستدعاء الدالة Engine.search(dataset, model, query, ...).

2. يقوم المحرك بتحميل حزمة (\$Bundle\$) مجموعة البيانات بشكل كسول (\$Lazily\$) الفهرس المقلوب الخاص بـ SQLite ، مخازن المتجهات الكثيفة المتوفرة، وأداة تحسين الاستعلام. يتم كاش (تخزين مؤقت) هذه الحزم بحيث تحدث هذه العملية مرة واحدة فقط.

3. في حال تفعيل ميزة التحسين، تقوم أداة QueryRefiner بتصحيح الأخطاء الإملائية و/أو توسيع الاستعلام — مع الاحتفاظ فقط بالتصحيحات/المرادفات الموجودة فعلياً في مجموعة الوثائق. (df > 0)

4. دالة **build_context** تقوم بتهيئة الاستعلام (الفعلي) باستخدام نفس أنبوب المعالجة الأولية (\$Preprocessing\ Pipeline) المستخدم لبناء الفهرس، لينتج عن ذلك كائن QueryContext يحتوي على النص الخام + الرموز. (\$Tokens)

5. يوجه المحرك الطلب للنموذج المحدد:

○ نموذج **TF-IDF / BM25** يحسب النتائج عبر دفع قوائم النشر الخاصة بالحدود المطابقة مباشرة من SQLite.

○ نموذج **BERT / Word2Vec** يقوم بترميز الاستعلام وتشغيل بحث جيب التمام عبر مصفوفة المتجهات المحملة في الذاكرة.

○ النماذج **الهجينة (\$Hybrids)** إما أن تدمج ترتيب عدة نماذج (بالتوازي) أو تقوم بالاسترجاع أولاً ثم إعادة الترتيب (بالتسلسل).

6. يتم ربط معرفات الوثائق الناتجة doc_id - **نصوصها الأصلية الكاملة** (المفكوك ضغطها من حقول BLOB في SQLite) وإعادتها ككائنات SearchResult مرتبة.

الجزء الخامس — تصميم التخزين والضغط

هناك مطلبان للتخزين يشكّلان طبقة البيانات، وتم استيفاؤهما بالكامل:

- **الوثائق الأصلية تُخزن بالكامل داخل قاعدة البيانات:** يخزن جدول docs في SQLite النص الأصلي الكامل لكل وثيقة كـ **BLOB مضغوط بصيغة zlib** (بدون أي بتر للنص). يتم تقديم نتائج الـ top-k مباشرة من قاعدة البيانات، وتعرض واجهة المستخدم الوثيقة بأكملها مدمجة في الصفحة.
- **النماذج والفهارس تُخزن بشكل مضغوط:** يتم تدقيق المتجهات الكثيفة مضغوطة بصيغة gzip (*vectors.f32.gz)، كما تُضغط ملفات معرفات الوثائق الجانبية (*docids.json.gz) مصفوفة Word2Vec عبارة عن ملف npz مضغوط، وقاموسها عبارة عن ملف JSON مضغوط بصيغة gzip. كما أن البيانات النصية الضخمة للفهرس المقلوب تُضغط بـ zlib داخل قاعدة البيانات.
- **لا يتم تدريب أي شيء أثناء وقت الاستعلام (\$Query\ Time):** يتم احتساب نتائج TF-IDF و BM25 من فهرس مبني مسبقاً (حيث يتم حساب IDF والمعايير لكل وثيقة مرة واحدة وقت البناء). يتم بناء المتجهات الكثيفة ونموذج Word2Vec دون اتصال (\$Offline) ويتم تحميلها فقط وقت الاستعلام. أول استعلام كثيف يقوم بفك ضغط ملف المتجهات إلى الذاكرة مرة واحدة، لتصبح عمليات الحساب اللاحقة سريعة جداً.

الجزء السادس — مخطط المجلدات (\$Directory\ Layout)

```
# ir_core/ محرك استرجاع المعلومات (مكتبة قابلة للاستيراد)
# text/ المعالجة الأولية (الوثائق + الاستعلامات)
# data/ تدقيق مجموعات البيانات/qrels + الاستعلامات
# index/ الفهرس المقلوب + (SQLite) مخزن المتجهات الكثيفة
# representation/ نماذج tfidf, bm25, word2vec, embeddings, hybrid
# query/ المعالجة + تحسين الاستعلام
```

```
# eval/ المقاييس + مشغل التقييم
# engine.py المنسق الموحد (Facade) المستخدم من الخدمات والواجهة الرسومية
# services/ بنية SOA: المعالجة/الفهرسة/الاسترجاع/التقييم/التحسين + البوابة
# ui/app.py واجهة مستخدم Streamlit
# scripts/ سكريبتات build_indexes.py, evaluate.py, run_full_eval.py, smoke_test.py
# docs/ ملفات ARCHITECTURE.md, REPORT.md, CODE_REFERENCE.md
# (المجلد الرئيسي # سكريبتات preprocessing.py, process_docs.py, check_ready.py)
# download_dataset.py, ir_win_fix.py
```

الجزء السابع — مرجع الكود: كل ملف ودالة

يوثق هذا الجزء كل ملف مصدري وكل دالة، وصنف، وتابع، ومسار HTTP يحتوي عليه النظام، مصنفة حسب الطبقة.

القسم أ — الإعدادات والأنواع المشتركة

ir_core/__init__.py

الهدف: علامة حزمة لمكتبة محرك ir_core؛ يعلن عن التوثيق الخاص بالحزمة وإصدارها.

لا يحتوي هذا الملف على أي دالات أو أصناف. يقتصر فقط على تحديد نص التوثيق للموديول) والذي يصف المكتبة بأنها محرك IR مجزأ يحتوي على تمثيلات متعددة للوثائق، وفهرسة مدعومة بالقرص، ومعالجة/تحسين الاستعلامات، والترتيب والتقييم (ويعرّف ثابتاً واحداً على مستوى الموديول وهو `__version__ = "1.0.0"`.

ir_core/config.py

الهدف: مركز الإعدادات الرئيسي: سجل مجموعات البيانات، مسارات الفهارس على القرص، عناوين روابط الخدمات المصغرة، والمعاملات الفائقة (\$Hyper-parameters\$) الافتراضية للنماذج والاسترجاع، بحيث لا يتم ترميز أي مسار أو معرف بشكل صلب في مكان آخر.

الثوابت على مستوى الموديول) PROJECT_ROOT: جذر المستودع)، INDEX_ROOT: المجلد الجذري لجميع الفهارس المحفوظة)، DATASETS قاموس يربط مفاتيح المجموعات بكائنات DatasetSpec، حالياً يحتوي على "argsme" فقط)، SAMPLE حزمة عينة مصغرة من 5 وثائق لاختبارات الدخان)، BERT_MODEL_NAME اسم نموذج (sentence-transformers، DEFAULT_TOP_K = 10، و SERVICE_URLS قاموس لعناوين المضيف والمنافذ للخدمات المصغرة).

الصنف DatasetSpec

صنف بيانات مجمد (\$Frozen\ Dataclass\$) يصف مجموعة وثائق معينة: أين تقع ملفات نصوصها المعالجة وكيفية جلب استعلامات التقييم وال qrels الخاصة بها من مكتبة ir_datasets. الحقول key: str: المعرف القصير المستخدم في الروابط/الواجهة)، name: str الاسم الصديق للمستخدم)، processed_json: str مسار مصفوفة JSON للوثائق المعالجة)، ir_datasets_id: str المعرف الخاص بمكتبة ir_datasets، و query_text_attrs: tuple (خصائص النص التي يتم محاولة قراءتها بالترتيب لاستخراج نص الاستعلام).

index_dir(self) -> str

خاصية للقراءة فقط تعيد مجلد الفهرس الخاص بمجموعة البيانات هذه، ويتم احتسابه عبر `os.path.join(INDEX_ROOT, self.key)` لا تقوم بإنشاء المجلد بل تركب المسار فقط.

الصف BM25Params

صنف بيانات يحمل المعاملات الفائقة الافتراضية لترتيب `BM25: k1: float = 1.5` (تشبع تكرار المصطلح) و `b: float = 0.75` (تطبيع طول الوثيقة).

الصف Word2VecParams

صنف بيانات للمعاملات الفائقة الافتراضية لنموذج `Word2Vec skip-gram` تشمل الأبعاد `dim: int = 100` النافذة `window: int = 5`، الحد الأدنى للتكرار `min_count: int = 2`، العينات السلبية `negatives: int = 5` الدورات `epochs: int = 3`، حجم الدفعة `batch_size: int = 1024`، ومعدل التعلم `lr: float = 0.025`.

`def get_dataset(key: str) -> DatasetSpec`

يقوم بتحويل مفتاح مجموعة البيانات إلى كائن `DatasetSpec` الخاص به. يعيد `SAMPLE` إذا كان المفتاح يطابق مفتاح العينة؛ وإلا يبحث عنه في `DATASETS`. يرفع خطأ `KeyError` في حال كان المفتاح غير معروف.

`def list_datasets() -> list[dict]`

يعيد قاموساً ملخصاً لكل مجموعة بيانات مسجلة، يحتوي على `key` و `name` (`available`) إذا كان ملف `processed_json` موجوداً على القرص).

`def ensure_index_dir(dataset_key: str) -> str`

يحدد مجلد الفهرس لمجموعة البيانات، ويقوم بإنشائه عبر `(os.makedirs(..., exist_ok=True))` ثم يعيد المسار، لضمان وجود المجلد قبل كتابة أي ملفات.

`ir_core/types.py`

الهدف: يعرف أنواع بيانات مشتركة صغيرة مستخدمة عبر طبقة الاسترجاع.

الصف SearchResult

صنف بيانات يمثل نتيجة بحث واحدة مرتبة. الحقول `doc_id: str`، `score: float`، `rank: int = 0`، `raw_text: str = ""` و `components: dict` (تُملأ أثناء الدمج الهجين بالأجزاء التفصيلية للنتائج من كل نموذج لدعم الشفافية).

`to_dict(self) -> dict`

يحول النتيجة إلى قاموس متوافق مع صيغة `JSON`. يقوم بتقريب النتيجة `score` وقيم `components` إلى 6 أرقام عشرية للحفاظ على أحجام `Payload` مدمجة ومحددة.

القسم ب — طبقة البيانات والنصوص

`ir_core/data/corpus.py`

الهدف: توفير وصول تدفقي إلى مجموعات الوثائق المعالجة أولاً والتي قد تصل أحجامها إلى عدة جيجابايت، بالإضافة إلى استعلامات التقييم وأحكام الصلة (`$qrels$`) من `ir_datasets`؛ حيث يتم تدفيق مصفوفات `JSON` المعالجة باستخدام مكتبة `json` بدلاً من تحميلها بالكامل في الذاكرة.

الصف Doc

صف بيانات يمثل وثيقة واحدة معالجة أولياً. الحقول: doc_id: str, tokens: list[str], raw_text: str.

def iter_docs(spec: DatasetSpec, limit: Optional[int] = None) -> Iterator[Doc]

يقوم بتدفيق الوثائق واحدة تلو الأخرى من ملف spec.processed_json بحسب ترتيب الملف. يفتح الملف بالنمط الثنائي ويستخدم json.items(f, "item") للتكرار بشكل كسول، ويعيد كائن Doc لكل سجل. يتم قراءة الحقول بشكل دفاعي، ويتوقف التدفق عند الوصول إلى الحد limit.

def count_docs(spec: DatasetSpec, limit: Optional[int] = None) -> int

يحسب عدد الوثائق عبر التكرار باستخدام iter_docs(...). يحسب عدد الوثائق عبر التكرار باستخدام iter_docs(...). يحسب عدد الوثائق عبر التكرار باستخدام iter_docs(...).

def _query_text(query_obj, attrs: tuple) -> str

دالة مساعدة خاصة تستخرج نص الاستعلام من كائن استعلام ir_datasets (والذي يختلف شكله بحسب مجموعة الوثائق). تجرب كل خاصية بالترتيب لتعيد أول نص متاح.

def load_queries(spec: DatasetSpec, limit: Optional[int] = None) -> list[tuple[str, str]]

تحميل استعلامات التقييم على شكل توابل (\$Tuples\$) من نمط (query_id, query_text). تستورد ir_win_fix ثم ir_datasets لتتكرر عبر الاستعلامات وتتوقف عند الحد limit.

def load_qrels(spec: DatasetSpec) -> dict[str, dict[str, int]]

تحميل أحكام الصلة كقاموس متداخل بالشكل {query_id: {doc_id: relevance}}.

ir_core/text/preprocessing.py

الهدف: أنبوب تهيئة النصوص القياسي والمشارك بين الوثائق والاستعلامات، مما يضمن تواجدهما في نفس الفضاء الحدودي؛ يعيد إنتاج نفس الأنبوب المستخدم لبناء ملفات JSON الخاصة بمجموعات البيانات.

عند الاستيراد، يضمن وجود حزم NLTK المطلوبة ويقوم بتحميلها صامتاً في حال غيابها، ثم يبني كائنات مفردة على مستوى الموديول (Stemmer): أداة Porter للجذوع (،) (LEMMAZER أداة WordNet للرد إلى الأصول)، و (STOPWORDS_EN كلمات التوقف الإنجليزية).

def _wordnet_pos(treebank_tag: str)

تحول وسم أجزاء الكلام (\$POS\ Tag\$) الخاص بـ Penn Treebank إلى الثوابت المتوافقة مع WordNet لتحسين عملية الرد إلى الأصول (\$Lemmatization\$).

def process(text: str, *, lowercase: bool = True, remove_punctuation: bool = True, remove_stopwords: bool = True, do_lemmatize: bool = False, do_stemming: bool = True) -> list[str]

المصدر الوحيد للحقيقة لتحويل النصوص إلى رموز (\$Tokens\$)، وتطبق بشكل متطابق على الوثائق والاستعلامات. الإعدادات الافتراضية (تفعيل التجذير \$Stemming\$ وإيقاف الرد للأصول (\$Lemmatization\$) تطابق طريقة بناء ملفات JSON للمجموعات، مما يضمن توافق فضاء الحدود بين الاستعلام والفهرس.

def stem(term: str) -> str

يجذر مصطلحاً واحداً (تحويل لحروف صغيرة + أداة (Porter) تُستخدم عند توسيع الاستعلام لضمان أن المرادفات المضافة تقع في نفس الفضاء المجذر للفهرس.

ir_core/query/processing.py

الهدف: يطبق معالجة الاستعلام (المتطلب #4) عبر تهيئة الاستعلام الخام بنفس الأنبوب الخاص بالوثائق وتغليفه في كائن QueryContext.

def build_context(raw_query: str, bm25_k1: Optional[float] = None, bm25_b: Optional[float] = None) -> QueryContext

يبنى كائن QueryContext: يخزن الاستعلام الخام، ويقوم بتحويله إلى رموز عبر دالة process بالرايات الافتراضية، ويمرر معاملات BM25 الاختيارية المحددة لكل استعلام.

ir_core/query/refinement.py

الهدف: يطبق تحسين الاستعلام (المتطلب #5) — تصحيح الإملاء، التوسيع بمرادفات WordNet ، اقتراح الاستعلامات، والتخصيص بناءً على سجل البحث — بطريقة واعية بمجموعة الوثائق (\$Corpus-aware\$) حيث لا يتم الاحتفاظ بالتصحيات أو مصطلحات التوسيع إلا إذا كانت موجودة بالفعل في الفهرس. ($df > 0$)

الصنف RefinementResult

صنف بيانات يجمع مخرجات عملية التحسين. الحقول تشمل: النص الأصلي original، المصحح corrected، النص المحسن الخام (refined_raw لنظام (BERT ، والرموز المحسنة refined_tokens (للاسترجاع اللفظي اللغوي)، بالإضافة إلى القواميس التي توضح التغييرات المطبقة.

to_dict(self) -> dict

يحول الحقول الثمانية إلى قاموس عادي لردود JSON.

الصنف QueryRefiner

أداة تحسين واعية بمجموعة الوثائق تستخدم كائن InvertedIndex للتحقق من وتصنيف التصحيحات، التوسيعات، والاقتراحات المحتملة بناءً على مفردات المفهرس الفعلية.

def __init__(self, index: InvertedIndex)

يخزن الفهرس InvertedIndex لاستشارته في التكرارات الحدودية وعينات المفردات.

def _best_correction(self, word: str) -> Optional[str]

يختار أفضل تصحيح إملائي لكلمة واحدة بناءً على مسافة التعديل (\$Edit\ Distance\$) بشرط أن يكون للمصطلح المختار تكرار مستندي في الفهرس $df > 0$.

def correct(self, query: str) -> tuple[str, dict]

يصحح إملاء استعلام بأكمله، ويعيد الاستعلام المصحح وقاموساً بالتغييرات المطبقة.

def expand(self, query: str, max_per_word: int = 2) -> list[str]

يضيف ما يصل إلى max_per_word من مرادفات WordNet لكل كلمة، بشرط أن تكون جذوع المرادفات موجودة في الفهرس $df > 0$ ولم تكن موجودة مسبقاً في الاستعلام.

```
def suggest(self, query: str, history: Optional[list[str]] = None, limit: int = 5) -> list[str]
```

يقدم ما يصل إلى limit من الاقتراحات: يبدأ بالاستعلامات السابقة من السجل history التي تشترك في رمز مع الاستعلام الحالي، ثم يكمل بالاستكمال التلقائي للكلمة الأخيرة عبر دالة index.vocab_sample(...).

```
def history_terms(self, history: Optional[list[str]], query: str, max_terms: int = 3) -> list[str]
```

يستخرج أبرز الكلمات من الاستعلامات السابقة ذات الصلة لانحياز عملية الاسترجاع نحو اهتمام المستخدم.

```
def refine(self, query: str, *, do_spell: bool = True, do_expand: bool = True, use_history: bool = True, history: Optional[list[str]] = None, max_per_word: int = 2) -> RefinementResult
```

المنسق الشامل لعملية التحسين بضربة واحدة؛ يطبق التصحيح، التوسيع، والتخصيص بحسب الرايات الممررة، ويبنى النصوص والرموز المحسنة لكلا نوعي الاسترجاع.

القسم ج — طبقة الفهرسة

ir_core/index/inverted_index.py

الهدف: فهرس مقلوب مدعوم بالقرص ومبنى على SQLite، يقدم تدقيقاً لمجموعة الوثائق إلى القرص ويعمل كركيزة مشتركة لقوائم النشر، وتكرار الوثائق، وقيم IDF، ومعايير TF-IDF لكل وثيقة (لكل من أدوات حساب نتائج TF-IDF/VSM و BM25).

```
def _db_path(spec: DatasetSpec) -> str
```

يعيد المسار المطلق لملف قاعدة بيانات SQLite (inverted_index.sqlite) داخل مجلد الفهرس الخاص بمجموعة البيانات.

```
def _compress_text(text: str) -> Optional[bytes]
```

يضغط النص الكامل للوثيقة بصيغة zlib بترميز UTF-8 والمستوى 6 (ليتم تخزينه كـ BLOB مدمج دون أي بتر للنص).

```
def _decompress_text(value) -> str
```

عكس دالة _compress_text: يفك ضغط كائن BLOB مضغوط بـ zlib ليعيده إلى نص عادي.

```
def _connect(path: str) -> sqlite3.Connection
```

يفتح اتصال قاعدة بيانات SQLite تم ضبطه خصيصاً لرفع كفاءة عمليات البناء والقراءة الضخمة. يتضمن إعدادات مثل تفعيل وضع WAL، وتهيئة حجم كاش يصل إلى 200 ميجابايت، وتسجيل دالات LOG و SQRT لتعمل حسابات الأوزان داخل SQL بنفس دلالات اللوغاريتم الطبيعي المستخدمة في دالات بايثون.

الصنف Posting

صنف بيانات يمثل سجلاً واحداً في قائمة النشر (\$Posting\$) مدمجاً مع البيانات الوصفية للوثيقة doc_rowid: int، tf: int، length: int، tfidf_norm: float. حمل المعيار (Norm) مع السجل يغني عن عملية بحث إضافية أثناء حساب النتائج.

الصنف InvertedIndex

واجهة القراءة والبناء الفهرسي فوق قاعدة بيانات SQLite لمجموعة بيانات معينة.

```
def __init__(self, spec: DatasetSpec)
```

يخزن مواصفات الحزمة، ويجهز كاش الاتصال الكسول دون إجراء عمليات إدخال/إخراج (\$I/O\$) هنا.

```
conn(self) -> sqlite3.Connection (خاصية)
```

يفتح اتصال القراءة المخزن مؤقتاً بشكل كسول (ويرفع خطأ FileNotFoundError مع تلميح للبناء إذا كان الملف مفقوداً).

```
def exists(self) -> bool
```

يعيد ما إذا كان ملف SQLite موجوداً على القرص دون فتح اتصال فعلي، ليستخدمه المحرك للتحقق من إمكانية استعمال الحزمة.

```
def close(self)
```

يغلق الاتصال المخزن مؤقتاً ويصفر المتغير لإتاحة إعادة الفتح لاحقاً.

```
def build(self, limit: Optional[int] = None, keep_raw: bool = True, log_every: int = 5000) -> dict
```

يبني الفهرس بأكمله من الصفر عبر تدقيق مجموعة البيانات (عملية تكرارية آمنة — \$Idempotent\$ تحذف أي ملفات قائمة أولاً). الخطوات: (1) إنشاء جداول الميتا والوثائق والحدود وقوائم النشر؛ (2) التكرار عبر الوثائق، تعيين معرف رقمي داخلي rowid يبدأ من 1، حساب تكرارات الرموز وطول الوثيقة، وضغط النص الخام؛ (3) حساب قيم df لكل حد بضربة واحدة عبر GROUP BY؛ (4) احتساب قيم $idf = \ln(N/df)$ وحفظها؛ (5) إنشاء فهرس شجرية B-tree؛ (6) حساب معيار L2 لكل وثيقة؛ (7) كتابة جدول الميتا وتشغيل ANALYZE. يعيد قاموس الميتا الناتج.

```
def df(self, term: str) -> int
```

يعيد التكرار المستندي (\$Document\ Frequency\$) لمصطلح معين من جدول الحدود (0 إذا لم يكن موجوداً).

```
def idf(self, term: str) -> float
```

يعيد قيمة IDF المحسوبة مسبقاً والممهدة ($\ln(N/df)$) للحد المحدد.

```
def postings(self, term: str) -> list[Posting]
```

يجلب جميع سجلات النشر للحد المحدد ككائنات Posting مدمجة ببيانات طول الوثيقة والمعيار. هذه هي الدالة الأساسية في الحلقة الداخلية لكلا نموذجي الاسترجاع اللفظي.

```
def get_docs(self, rowids: Iterable[int]) -> dict[int, dict]
```

يجلب البيانات النصية المقابلة للمعرفات الداخلية rowid مع فك ضغط النصوص لعرضها في نتائج البحث اللفظي.

```
def get_docs_by_id(self, doc_ids: Iterable[str]) -> dict[str, str]
```

يربط معرفات الوثائق الخارجية بنصوصها المفكوك ضغطها، ويقسم عملية البحث في جملة N ضمن مجموعات من 500 عنصر. تستخدمها نماذج التضمينات الكثيفة التي تعتمد على معرفات النصوص.

```
def rowids_for(self, doc_ids: Iterable[str]) -> dict[str, int]
```

يحول المعرفات الخارجية doc_id إلى معرفات rowid الداخلية. يسمح لنماذج إعادة الترتيب اللفظي بتقييد نطاق عملها على مرشحي المرحلة الأولى فقط.

```
def vocab_sample(self, prefix: str, limit: int = 10) -> list[str]
```

يجلب عينة من المفردات التي تبدأ بالبائدة prefix مرتبة تنازلياً حسب التكرار لدعم ميزة الاقتراحات التلقائية.

ir_core/index/vector_store.py

الهدف: مخزن متجهات كثيفة مدعوم بالقرص لتمثيلات التضمينات؛ حيث يتم حفظ المتجهات على القرص كملفات float32 مطبوعة بمقيار L2 ومضغوطة بـ gzip ، وأثناء وقت البحث، يتم فك ضغطها في الذاكرة لمرة واحدة وحساب النتائج عبر عمليات حاصل الضرب النقطي المصفوفي (\$Matrix-Vector \ Dot \ Product\$) السريعة (حيث يتطابق الضرب النقطي مع جيب التمام نتيجة التطبيع المسبق).

الصف VectorStoreWriter

يقوم بتدقيق متجهات float32 إلى القرص أثناء عملية البناء؛ يستدعي الباني الدالة add لكل دفعة ثم close.

```
def add(self, doc_ids: list[str], vectors: np.ndarray)
```

يضيف دفعة متجهات: يحولها إلى float32 ، يطبع كل سطر بمقيار L2 (مع الحماية من المعايير الصفرية)، يكتب البايتات الخام إلى دفق gzip ، ويحفظ المعرفات.

```
def close(self)
```

يغلق دفق متجهات gzip ، ويكتب ملف المعرفات الجانبي المضغوط، وملف الميتا العادي بصيغة JSON.

الصف VectorStore

واجهة للقراءة فقط فوق مخزن المتجهات المبني لاستخدامه في بحث جيب التمام.

```
def exists(self) -> bool
```

يعيد True فقط إذا كانت الملفات الثلاثة (المتجهات، الميتا، ومعرفات الوثائق) موجودة على القرص.

```
def score_subset(self, query_vec: np.ndarray, doc_ids: list[str]) -> dict[str, float]
```

يحسب تشابه جيب التمام للاستعلام مقابل مجموعة جزئية محددة من الوثائق (مخصصة لإعادة الترتيب في النظام الهجين المتسلسل).

```
vectors(self) -> np.ndarray (
```

يقوم بفك ضغط المصفوفة الكاملة بأبعاد (n, dim) لمرة واحدة في الذاكرة عبر

np.frombuffer(...).reshape(...) ويقوم بكاش لها. تكلفة فك الضغط تدفع لمرة واحدة عند أول طلب.

```
def search(self, query_vec: np.ndarray, top_k: int = 10, batch: int = 100_000) -> list[tuple[str, float]]
```

البحث الشامل بالقوة الغاشمة (\$Brute-force\$) عن أعلى k نتائج. يعالج المصفوفة في دفعات أسطر، ويستخدم دالة `linalg.partition` لتقليص مرشحي كل دفعة بشكل مستمر لضمان استهلاك ذاكرة محدود، ثم يفرز النتائج النهائية تنازلياً ليعيد الحزمة `[(doc_id, score)]`.

القسم د — طبقة التمثيلات والمطابقة

`ir_core/representation/__init__.py`

الهدف: علامة حزمة توثق أن جميع نماذج التمثيلات وحساب النتائج تعرض واجهة موحدة بالشكل `search(query_tokens, top_k, ...)` -> `list[SearchResult]` والدمج الهجين. يحتوي على نص التوثيق فقط.

`ir_core/representation/base.py`

الهدف: يعرف الواجهة الموحدة للمسترجعات عبر كائن `QueryContext` الذي يحمل الاستعلام الخام ورموزه معاً، بالإضافة إلى مغلفات `LexicalRetriever` و `DenseRetriever`.

الصف `LexicalRetriever`

مغلف موحد للنموذجين اللفظيين اللغويين (TF-IDF) أو (BM25)، حيث يستهلك كلاهما الرموز والفهرس المقلوب المشترك.

```
def search(self, ctx: QueryContext, top_k: int = 10) -> list[SearchResult]
```

يوجه الطلب إلى نموذج TF-IDF عندما يكون الاسم المختار هو "tfidf"، وإلى نموذج BM25 فيما عدا ذلك.

```
def rescore(self, ctx: QueryContext, doc_ids: list[str]) -> dict[str, float]
```

يعيد حساب نتائج مجموعة مرشحة محددة (للنظام الهجين المتسلسل) عبر تقييد محرك الحساب على المعرفات الرقمية المحددة فقط.

الصف `DenseRetriever`

مغلف موحد لتمثيلات التضمينات يعرض نفس الواجهة البرمجية لـ `LexicalRetriever`. يقوم بتمرير الاستعلام الخام (الأصلي) إلى النماذج الكثيفة لأن شبكات الترميز تعمل على النصوص مباشرة وليس الرموز المقطعة.

`ir_core/representation/tfidf.py`

الهدف: نموذج فضاء المتجهات (\$VSM\$) مع حساب الأوزان بطريقة (TF-IDF (ltc وتشابه جيب التمام (المتطلب #2، النموذج 1)؛ لا يتم تمثيل متجهات الوثائق بالكامل في الذاكرة — بل يتم تجميع حاصل الضرب النقطي حداً بحد عبر سجلات النشر وقسمته على المعايير المحسوبة مسبقاً لكل وثيقة وللاستعلام.

```
def search(self, query_tokens: list[str], top_k: int = 10, candidate_filter: Optional[set[int]] = None) -> list[SearchResult]
```

يحسب تشابه جيب التمام بين الاستعلام وكل وثيقة تحتوي على حد من حدود الاستعلام. يبني أوزان الاستعلام، ويتنقل عبر قوائم النشر للحدود ليجمع حاصل الضرب النقطي، ثم يطبع الناتج بقسمته على معيار الوثيقة ومعيار الاستعلام ليعيد القائمة مدمجة بالنصوص الأصلية.

ir_core/representation/bm25.py

الهدف: محرك حساب النتائج الاحتمالي BM25/Okapi (المتطلب #2، النموذج 3) الذي يعمل على نفس الفهرس المقلوب لـ TF-IDF ، ويتيح تخصيص معاملات الضبط k1 و b لكل استعلام على حدة.

```
def search(self, query_tokens: list[str], top_k: int = 10, k1: Optional[float] = None, b: Optional[float] = None, candidate_filter: Optional[set[int]] = None) -> list[SearchResult]
```

يحسب النتائج وفق معادلة BM25 Okapi القياسية. يستخرج قيم المعاملات الفعالة، ويحسب قيمة IDF لكل حد فريد عبر صيغة Robertson/Spärck-Jones ، ويتنقل عبر قوائم النشر ليطبق معادلة تشبع التكرار وتطبيع الطول بناءً على طول الوثيقة الحالي ومعدل الأطوال العام. avgdl.

ir_core/representation/embeddings.py

الهدف: يمثل نموذجي التضمينات الكثيفة (المتطلب #2، النموذج 2): مشفر BERT المعتمد على حزمة-sentence transformers، ومشفر Word2Vec المبني على PyTorch. يتضمن دالات البناء التي تدفق الوثائق إلى كائن VectorStore، وصنف EmbeddingRetriever المشترك الذي يرمز الاستعلام ويشغل بحث جيب التمام.

الصنف BertEncoder

مغلف كسول حول نموذج sentence-transformers يؤجل عملية تحميل الشبكة في الذاكرة حتى أول استخدام فعلي لها. دالة encode_texts تقوم بترميز النصوص إلى مصفوفة float32.

الصنف Word2VecEncoder

يكيف نموذج Word2VecModel المدرب ليعمل كمشفّر؛ حيث يقوم بتقطيع نص الاستعلام الخام باستخدام دالة process الموحدة ويعيد متوسط ناقلات الكلمات المكونة له والتي تقع ضمن قاموس النموذج.

```
def build_word2vec_index(...) و def build_bert_index(...)
```

دالات بناء تقوم بتمرير نصوص مجموعة البيانات بالكامل على دفعات عبر المشفرات المحددة، وتقوم بحفظ النتائج مطبوعة بمعايير L2 مباشرة إلى كائنات VectorStoreWriter على القرص.

ir_core/representation/word2vec.py

الهدف: نموذج Word2Vec من نوع skip-gram حقيقي مع ميزة العينات السلبية (\$Negative\ Sampling\$) مبني مباشرة بلغة Python نظراً لعدم توفر حزم gensim جاهزة لنسخة بايثون (3.14)؛ يتم تمثيل الوثائق والاستعلامات كمستوسط ناقلات الكلمات داخل القاموس. هذا هو نموذج التضمين الثاني إلى جانب BERT.

الصنف Word2VecModel

يحمل مصفوفة الكلمات المدربة بأبعاد (V, dim) وقاموس الحدود، ويتولى عمليات تحويل قوائم الرموز إلى متجهات.

```
def embed_tokens(self, tokens: Iterable[str]) -> np.ndarray
```

يحول قائمة رموز إلى متجه واحد عبر حساب متوسط متجهات الكلمات المعروفة للقاموس، ويسقط الكلمات الغريبة (\$OOV\$). يعيد متجهاً صفرياً إذا غابت كل الكلمات عن القاموس.

```
def train_word2vec(token_stream_factory, ...)
```

يدرب نموذج skip-gram من الصفر عبر دورتين على مجموعة البيانات. الدورة الأولى تبني القاموس وتصفية الكلمات النادرة وتحسب توزيع العينات السلبية المقدر بـ $\text{freq}^{0.75}$ الدورة الثانية تمرر التجميعات عبر صنف الـ Network الصغير المسمى SkipGram المكون من جدولتي تضمين للمركز والسياق، ويحدث الأوزان باستخدام خوارزمية Adam وتحسين دالة الخسارة المعتمدة على الـ logsigmoid للعينات الموجبة والسلبية معاً، ليقوم أخيراً بتطبيع المصفوفة وحفظها.

ir_core/representation/hybrid.py

الهدف: تطبيق تمثيل الاسترجاع الهجين (المتطلب #2، النموذج 4) بكلا أسلوبيه — الدمج المتوازي) الجمع الموزون أو دمج الرتب المتبادلة (RRF) والتسلسل المستندي المتدرج، مع الاحتفاظ بتفاصيل نتائج النماذج المكونة في كل نتيجة.

def parallel_hybrid(...)

يشغل كل مسترجع بشكل مستقل تماماً ليُجلب كل منهم مسبقاً (Pool) من المرشحين. في حال اختيار دمج fusion $\text{w} \times \frac{1}{60}$ ، يتم حساب النتائج بناءً على رتبة الوثيقة في كل نموذج وفق الصيغة rank (الأسلوب الموزون (weighted)، يتم تطبيع نتائج النماذج المختلفة إلى النطاق $[0, 1]$ باستخدام طريقة min-max وتجميع حاصل ضربها في أوزان النماذج المحددة.

def serial_hybrid(stage1, stage2, ...)

شلال الاسترجاع المتسلسل؛ يقوم النموذج الأول (الرخيص والواسع الاستدعاء (High-recall) بجلب candidate_k من الوثائق، ثم يقوم النموذج الثاني (الأكثر تكلفة والأعمق دلالة (Semantic) بإعادة احتساب نتائج تلك الوثائق المحددة فقط عبر تابع rescore . يتم دمج النتائج مع منح الأولوية لنتائج المرحلة الثانية مع كسر التعادل بنسبة ضئيلة جداً من نتائج المرحلة الأولى.

ir_core/engine.py

الهدف: نقطة الدخول الموحدة والوحيدة لعمليات الاسترجاع والتحسين، والمستخدم من قبل كافة الخدمات المصغرة وواجهة مستخدم Streamlit؛ حيث تتولى تحميل حزم المجموعات بشكل كسول وتوجيه الطلبات للنماذج المفردة أو الهجينة. الكائن $\text{ENGINE} = \text{Engine}()$ يمثل المفردة المشتركة للنظام.

def search(self, dataset_key: str, model: str, query: str, *, top_k: int = 10, ...)

الموزع الرئيسي للنظام؛ يقوم بتحميل الحزمة المطلوبة، ويشغل أداة التحسين إذا طلبت الإعدادات ذلك لينتج عنها النص والرموز الفعالة الجديدة، ثم يبني كائن QueryContext ويوجه عملية البحث إلى المسترجع المفرد المقابل، أو يستدعي التوابع الخاصة بالأنظمة الهجينة parallel و serial . يعيد قاموساً متكاملًا يحتوي على معلومات الاستعلام الفعلي، تفاصيل التحسين إن وجدت، وقائمة النتائج المرتبة المسلسلة بدقة.

القسم ه — طبقة التقييم والمقاييس

ir_core/eval/metrics.py

الهدف: يطبق مقاييس كفاءة أنظمة استرجاع المعلومات القياسية (المتطلب #8). تستقبل كل دالة قائمة معرفات الوثائق المرتبة وقاموس أحكام الصلة للاستعلام المحدد، وتتعامل المقاييس الثنائية مع أي صلة $\text{score} > 0$ على أنها وثيقة ذات صلة، بينما يستخدم مقياس nDCG القيمة الرقمية المدرجة كحجم للمكسب.

def precision_at_k(ranked: list[str], qrel: dict[str, int], k: int = 10) -> float

الدقة عند العمق k : نسبة الوثائق ذات الصلة المتواجدة في أول k وثيقة مرتبة. القاسم دائماً هو k ، وبالتالي تُعاقب القوائم القصيرة التي تفشل في جلب عدد كافٍ من النتائج.

```
def recall(ranked: list[str], qrel: dict[str, int], k: int | None = None) -> float
```

الاستدعاء: نسبة الوثائق ذات الصلة التي تم جلبها من إجمالي كافة الوثائق المعروفة كذات صلة في الـ qrels.

```
def average_precision(ranked: list[str], qrel: dict[str, int], k: int | None = None) -> float
```

متوسط الدقة (AP) لاستعلام واحد — ويمثل متوسط قيم الدقة المحسوبة عند كل موقع يظهر فيه مستند ذو صلة. حساب متوسط هذه القيمة لجميع الاستعلامات ينتج عنه مقياس MAP الشامل للنظام.

```
def ndcg_at_k(ranked: list[str], qrel: dict[str, int], k: int = 10) -> float
```

مقياس المكسب التراكمي المخصص المنظم عند العمق k . يحسب الخصم اللوغاريتمي للوثائق بحسب موقعها ليعطي وزناً أعلى للصلة التي تظهر في الصدارة، ويقسم الناتج على القيمة المثالية الممكنة للاستعلام (IDCG) ليعيد قيمة محصورة بدقة في النطاق $[0, 1]$.

ir_core/eval/evaluate.py

الهدف: مشغل عمليات التقييم (المتطلب #8) الذي ينفذ نموذجاً معيناً عبر مجموعة من الاستعلامات المحكومة، ويحسب نتائج المقاييس لكل استعلام عبر موديول metrics ويجمعها في المتوسط، كما يدعم مقارنة كفاءة النظام قبل وبعد تطبيق ميزة التحسين.

```
def evaluate_model(...)
```

يقيم نموذجاً واحداً؛ يحلل المواصفات، ويحمل الاستعلامات وأحكام صلتها، ويصفي الاستعلامات غير المحكومة. لكل استعلام، يستدعي دالة البحث ENGINE.search بالعمق المحدد، ويستخرج المعارف ليحسب مقاييسها ويجمعها في متوسط عام يعاد كقاموس إحصائي شامل مدمج بتفاصيل كل استعلام.

```
def compare_refinement(...)
```

يشغل عملية التقييم مرتين على نفس النموذج لقياس الفارق بدقة: المرة الأولى مع إيقاف التحسين (before)، والمرة الثانية مع تفعيله (after). يحسب الفارق الرياضي Δ لكل مقياس (بعد - قبل) ليعيد تقريراً يوضح مدى مساهمة أدوات التحسين في رفع كفاءة الاسترجاع.

القسم و — سكريبتات واجهة السطر البرمجي (CLI)

scripts/build_indexes.py

الهدف: واجهة سطر برمجي لبناء كافة الفهارس والتمثيلات المقلوب، Word2Vec، و BERT لمجموعة بيانات محددة (المتطلب #3 و جزء البناء للمتطلب #2). يعتمد على التدقيق المستمر للبيانات ليتمكن من معالجة الملفات الضخمة دون استهلاك الذاكرة العشوائية. يضيف جذر المشروع إلى مسار النظام تلقائياً لضمان التعرف على حزمة ir_core.

```
def main()
```

نقطة الدخول المعتمدة على حزمة argparse. تعرّف الرايات البرمجية وتستقبل المدخلات لتستدعي دالة البناء التتابعية الموقوتة مسجلة تقدم العمل في كل خطوة. الرايات تشمل --dataset (إجباري)، --models (الافتراضي بناء الكل)،

limit--(الحد الأقصى للوثائق، الافتراضي 50000؛ و 0 يعني الكل)، بالإضافة إلى المعاملات الخاصة بـ Word2Vec وحجم دفعات BERT.

scripts/evaluate.py

الهدف: واجهة سطر برمجي لتقييم نموذج مفرد تطبع مقاييس MAP/Recall/P@10/nDCG@10 ككائن JSON ، وتدعم خيار المقارنة المباشرة للتأثير قبل وبعد التحسين. تتضمن رايات لتحديد العمق واختيار النموذج وتخصيص معاملات BM25.

scripts/run_full_eval.py

الهدف: أداة سطر برمجي تقوم بتقييم شامل لكافة النماذج المتاحة لمجموعة البيانات المحددة وتطبع جدولاً منسقاً بصيغة Markdown بالإضافة إلى تفريغ JSON كامل لتغذية ملف التقرير الخاص بالمشروع docs/REPORT.md (المتطلب #8). تتضمن إجبار ترميز الخرج على UTF-8 لتفادي انهيار السكريبت في مشغلات أوامر ويندوز القديمة عند طباعة رموز الدلتا. (\$\Delta\$)

scripts/smoke_test.py

الهدف: اختبار دخان متكامل يعمل من البداية إلى النهاية على عينة الـ 5 وثنائى المدمجة (بدون الحاجة لمكتبة ir_datasets). يقوم ببناء الفهارس المقلوبة ومتجهات Word2Vec و BERT المصغرة، ويصفر كاش المحرك ليجبره على إعادة التحميل، ثم يشغل عمليات البحث عبر النماذج المفردة والهجينة ويطبق أداة تحسين الاستعلام ليتحقق من سلامة كافة أجزاء النظام البرمجية ويطبّع في الختام "SMOKE TEST PASSED".

القسم ز — البنية الموجهة للخدمات(SOA)

ملخص تصميم بنية SOA: يتكون الجزء الخلفي من ستة عمليات FastAPI منفصلة وقابلة للنشر بشكل مستقل. خمسة منها تمثل خدمات النطاق التخصصية — المعالجة الأولية (8001)، الفهرسة (8002)، الاسترجاع (8003)، تقييم الترتيب (8004)، وتحسين الاستعلام (8005) — وكل منها مبني عبر دالة مصنع التطبيقات المشتركة common.make_app لتعرض تلقائياً الرابط الموحد GET /health وتفوض المهام لمكتبة ir_core المشتركة.

بوابة واجهة برمجيات التطبيقات (API Gateway) على المنفذ (8000) تمثل النقطة العامة الوحيدة المتاحة للعملاء وتطبق نمط الـ Gateway/Facade حيث لا تحتوي على منطق أعمال بل توجه الطلبات للخدمات الخلفية عبر عملاء REST مدمجين بصيغة بايثون القياسية، وتغلف أخطاء الخدمات الخلفية في هيئة خطأ HTTP 502 نظيف عبر دالة المراقبة safe_. كما تتولى عملية التنسيق المركب للرابط search_refined/.

services/common.py

الهدف: أدوات مشتركة للخدمات المصغرة: تشمل مصنع كائنات FastAPI ودالات إرسال طلبات الـ REST المعتمدة على مكتبة بايثون القياسية (urllib الدالتين post_json و get_json) لإبقاء الخدمات منفصلة وضعيفة الاقتران (\$Loosely Coupled) دون فرض مكتبات خارجية إضافية.

services/gateway.py

الهدف: بوابة الـ API للنظام (المنفذ 8000). تدير مسارات التحقق من صحة الخدمات المكتشفة تعيد توجيه طلبات البحث، والتحسين، والاقتراحات، والتقييم، والمقارنة. كما تنفذ مسار التنسيق المركب POST /search_refined الذي يستدعي خدمة التحسين أولاً، ويأخذ مخرجاتها النصية المحسنة ليغذي بها طلب خدمة الاسترجاع، ويعيد النتيجة مدمجة بالكامل.

services/preprocessing_service.py

الهدف: خدمة المعالجة الأولية (المنفذ 8001). تمتلك مسؤولية واحدة محددة: تحويل النصوص الخام إلى رموز مطهرة ومجذرة باستخدام نفس الأنبوب القياسي المطبق على الوثائق (المتطلب #4). الرابط: POST /process :

services/indexing_service.py

الهدف: خدمة الفهرسة (المنفذ 8002). تمتلك الفهرس المقلوب ومخازن المتجهات الكثيفة: تقدم تقارير عن حالة البناء للمجموعات، وتعرض مسارات لفحص المفردات والبادئات، وتسمح بطلب بناء جزئي مقيد بالحدود. الروابط GET : /datasets, GET /status/{dataset_key}, GET /vocab/{dataset_key}, POST /build.

services/retrieval_service.py

الهدف: خدمة الاسترجاع (المنفذ 8003). تمتلك منطق ومسؤولية مطابقة وترتيب الوثائق (المتطلب #6): تستقبل طلب البحث متضمناً اسم النموذج والمعاملات الفائقة وعبارات الاستعلام وخيارات الأنظمة الهجينة لتفويض المحرك ENGINE المشترك وتصيغ قائمة النتائج المرتبة. الرابط: POST /search :

services/ranking_eval_service.py

الهدف: خدمة التقييم وقياس الترتيب (المنفذ 8004). تمتلك مسؤولية القياس الفوقي لكفاءة النظام دون اتصال (المتطلب #8) عبر احتساب قيم إحصائيات مقاييس MAP/Recall/P@10/nDCG@10 لمجموعات البيانات، وإجراء المقارنات الثنائية لتأثيرات التحسين. الروابط: POST /evaluate, POST /compare :

services/query_refinement_service.py

الهدف: خدمة تحسين الاستعلامات (المنفذ 8005). تمتلك مسؤوليات المتطلب #5 بالكامل: تصحيح الأخطاء، التوسيع بالمرادفات، وتقديم الاقتراحات التلقائية بناءً على مفردات الفهرس وسجلات البحث. الروابط: POST /refine, POST /suggest :

services/run_all.py

الهدف: مشغل بيئة التطوير الذي يقوم بإطلاق الخمس خدمات المصغرة بالإضافة إلى البوابة معاً في وقت واحد، حيث يفتح كل منها كعملية فرعية مستقلة (\$Subprocess) تحت إدارة أداة uvicorn. يقوم بتأخير انطلاق البوابة لثنائية واحدة لضمان جاهزية الخدمات الخلفية، ويربط إشارة الإغلاق (Ctrl-C) بدالة إغلاق نظيفة تضمن إنهاء كافة العمليات التابعة بأمان قبل الخروج.

القسم ح — سكربتات الأدوات المساعدة في المجلد الرئيسي

preprocessing.py

الهدف: موديل معالجة النصوص المستقل المبني على NLTK (الأنبوب البرمجي الأصلي المستخرج من دفاتر الملاحظات والمسؤول عن توليد ملفات مصفوفات الوثائق الأولية). يتضمن الدالة المرنة الشاملة preprocess_text التي تقبل رايات لتفعيل أو إيقاف عمليات تحويل الحروف الصغيرة، إسقاط الترقيم، تصفية كلمات التوقف، وسم أجزاء الكلام، والرد للأصول، والتجذير بطريقة Porter. كما يقدم الدالة المغلفة المتوافقة رجعيًا clean_and_preprocess.

process_docs.py

الهدف: السكربت المسؤول عن التكرار عبر وثائق مكتبة `ir_datasets` واستخراج النصوص المفضلة لها وتمريضها عبر أنبوب مودبول المعالجة `preprocessing` لتصدير وحفظ النتائج في ملف JSON بهيكل مكون من المعرف والنص والرموز. يدعم تحديد عمق وعدد الوثائق المراد معالجتها عبر راية `--limit`.

`check_ready.py`

الهدف: سكربت فحص سريع للتحقق من أن مجموعة وثائق `Args.me` محملة محلياً وجاهزة للاستخدام. يسحب عدد الوثائق الإجمالي ويجلب استعلاماً واحداً لطباعة معرفه ونصه للتأكد من سلامة الربط.

`download_dataset.py`

الهدف: سكربت التحميل المسبق لمكونات مجموعة البيانات (الوثائق، الاستعلامات، وأحكام الصلة `qrels`) لضمان حفظها في كاش القرص وتفاذي عمليات التحميل الشبكي البطئ أثناء تشغيل خدمات النظام أو عمليات التقييم. يستدعي تلقائياً المعرف الخاص بـ `Args.me` المشتق لمهمة `Touché 2020` الأولى.

`ir_win_fix.py`

الهدف: سكربت حل مشكلات توافق نظام تشغيل ويندوز. يقوم بتبديل دالات ومؤشرات نظام الملفات والأذونات داخل بيئة عمل بايثون لتفاذي انهيار مكتبة `ir_datasets` عند محاولة فك ضغط الروابط والمجلدات الرمزية (`$Symlinks`) داخل بيانات ويندوز، مما يضمن عمل النظام بسلاسة وعبر المنصات المختلفة.